

Shared Structure, Independent Evidence

Alpha-normalised compression distance for refactoring and test design

Hara Renia Seihun

24 September 2026

Abstract

Two engineering decisions ask opposite questions about shared information. When two implementations encode the same mechanism under different local names, their shared structure suggests a refactoring. When a test encodes the implementation again, its shared structure suggests a correlated oracle: the same misunderstanding can make both agree. We describe **alpha-normalised Normalized Compression Distance (NCD)** as a practical attention signal for both decisions. Language-aware normalisation removes arbitrary naming differences; compression estimates remaining structural redundancy. For refactoring, the question is whether repeated information should have one owner. For testing, the question is whether an assertion contributes an independent behavioural constraint.

The central test-design principle is that a good test brings **surprising information with respect to the implementation**: a contract, boundary, independently obtained result, algebraic relation, or observable effect that is not merely a transcription of how the implementation works. We make “surprising” operational as discrimination among plausible faulty implementations, not textual novelty. NCD can expose structural mirroring; it cannot establish test quality by itself. A dependency-free Python reference tool and constructed, reproducible examples accompany this research note. The contribution is a unified engineering workflow, not a new compression distance, a semantic equivalence decision procedure, or a validated universal test-quality metric.

1. One measure, two questions

Suppose two functions walk a collection, select eligible elements, accumulate a field and return the total. Their argument names and local variables differ. A textual diff emphasises those names; a structural comparison reveals one mechanism. Moving that mechanism into one owner can reduce the number of places where a future correction must be made.

Now suppose a test computes its expected total by walking the same collection, selecting eligible elements using the same condition and accumulating the same field. The resemblance has a different meaning. If the eligibility condition is wrong in both places, the test passes. The test has duplicated the decision rather than independently constrained it.

The two cases suggest a paired rule:

Refactor information that has multiple implementation owners. Seek tests whose verdict has an independent source.

This does not mean all repetition is bad. Similar implementations can have distinct contracts and independent change schedules. A test must also name inputs and outputs that the implementation recognises. The useful distinction is between **shared mechanism**, **shared specification** and **shared mistake**.

The approach joins existing work on normalised code clones [3], compression similarity [1, 2], test oracles [4], metamorphic testing [5] and mutation analysis [6]. None of those ingredients is new. The proposal is to make their relationship an explicit review practice.

Compression-based test diversity was proposed by Feldt and colleagues [7] and developed into test-set diameter [8]. Ishio and colleagues used NCD to retrieve cloned buggy code [9]. Elgendi and colleagues evaluated test-source string distances, including NCD, for suite reduction using mutation-score retention [10]. Thus neither NCD for clone search nor NCD for test diversity originates here. The emphasis is the joint alpha-normalised workflow, especially scrutiny of oracle mechanisms against their implementations.

2. Representation before distance

2.1 Alpha-normalisation

In the lambda calculus, alpha-equivalence concerns consistent renaming of bound variables. In source analysis, a corresponding normalisation maps locally bound identifiers to canonical labels while preserving their binding relationships. For example:

```
def invoice_total(entries):
    total = 0
    for entry in entries:
        total += entry.amount
    return total
```

and the same function with `entries`, `total` and `entry` consistently renamed should have identical normalised representations. The use of one binding twice must remain distinguishable from the use of two different bindings. Replacing every identifier with a single token is not alpha-normalisation.

A useful conservative representation retains:

- operators, control flow and statement ordering;
- literal values, especially thresholds, units and boundary constants;
- external names, attributes and calls whose identities carry meaning;
- binding relationships and relevant type or signature structure.

Comments and formatting do not belong to the structural representation. Documentation and specifications can still be reviewed separately as sources of requirements.

A broader *shape* representation may also abstract literal values or API names. That can help discover analogous mechanisms across domains, but it is a separate, more lossy projection. Erasing `>=` versus `>`, a currency unit, or a cleanup call can erase precisely the distinction the engineer needs. Always retain original locations and source for review.

Even consistent local renaming is not automatically semantics-preserving in a reflective language. Python keyword arguments, `locals()`, inspection and string-based lookup can observe names. The

prototype is a structural analyser, not a source-to-source equivalence proof. It reports unsupported binding constructs rather than silently treating them as ordinary local variables.

2.2 Normalized Compression Distance

Let A be the chosen normalisation and let C measure compressed bytes. The familiar distance is

$$\text{NCD}(x, y) = \frac{C(xy) - \min(C(x), C(y))}{\max(C(x), C(y))}, \quad x = A(P), \quad y = A(Q).$$

Lower values indicate greater reuse of compressible structure. NCD is a computable compression-based approximation motivated by normalized information distance; it is not access to Kolmogorov complexity or program meaning.

An experiment must fix the serialisation, compressor, compression settings, concatenation convention and normalisation policy. Real compressors have headers, finite windows and order effects. Short functions can be dominated by overhead and shared AST syntax. Unequal unit sizes can distort comparisons. A finite-compressor score need not attain zero on identical inputs or obey metric axioms; clipping values into an ideal interval would conceal that behaviour.

Compare like-sized, like-kind units and retain their byte sizes. Use several interpretable anchors: a renamed duplicate, a related but different function and an unrelated function of similar size. A universal cutoff such as “below 0.3 means duplicate” is not justified by the definition.

2.3 Retrieval and scale

An all-pairs scan of n declarations requires $O(n^2)$ pair comparisons. Token-shingle overlap can cheaply retrieve candidates before compression. That changes recall: a filter that misses a candidate prevents NCD from ever evaluating it. Record the filter and expose its threshold. Approximate retrieval is useful engineering, not an exhaustive claim.

Classification should precede interpretation. Production implementations, test oracles, generated code and fixtures have different reasons to resemble each other. Do not mix them into one “delete redundancy” list. In the test workflow, compare the **expected-value or assertion mechanism** against the relevant implementation, rather than allowing imports, setup and assertion-library syntax to dominate a whole-file score.

3. Refactoring: find a common owner

3.1 Renamed duplicate folds

Consider two billing paths that sum amounts for records satisfying the same eligibility rule. After local names are normalised, duplicated control flow becomes visible. A useful refactoring makes the shared accumulation mechanism explicit and leaves business policy at the call site:

```
def sum_selected(rows, accepts, amount):
    total = 0
    for row in rows:
        if accepts(row):
            total += amount(row)
    return total
```

The important result is not fewer tokens. The eligibility policy and money representation must remain correct, and callers with genuinely different policies must keep those differences. If the two policies have independent owners, an abstraction that couples their future changes can be worse than two small functions. Similarity nominates a design question; contracts decide the answer.

3.2 Repeated effect protocols

Duplication also appears in protocols: acquire, run, commit or roll back, release. Locally different resource names can obscure the same cleanup structure. Giving the protocol one owner can prevent fixes to exceptional paths from landing in only one caller.

This is a stronger example than deduplicating arithmetic because ordering is observable. Before and after refactoring, check traces of success and failure, including a failure during the body. A function that returns the same value while leaking its resource is not behaviourally equivalent. The accompanying examples keep this acceptance criterion executable.

3.3 A review sequence

1. Extract comparable declarations and normalise them conservatively.
2. Rank candidate pairs, retaining paths, sizes and source context.
3. Identify the common mechanism and the policy differences.
4. Decide whether one owner improves future change, not just today's count.
5. Refactor and exercise independent value, boundary and effect contracts.

NCD is most useful before the design decision, when the hidden repetition is still hard to see. It is not a proof that extraction into a helper is the right architecture.

4. Tests: surprise must concern behaviour

4.1 The correlated-oracle failure

Suppose a published price contract says that an order at the threshold receives a discount. The implementation accidentally uses `amount > threshold`. A test which computes `expected` with the same `>` condition repeats the bug. Renaming variables or extracting the `expected-value` loop does not make the oracle independent.

Alpha-normalisation can reveal this otherwise disguised resemblance. The actionable diagnosis is not “the score is low, therefore delete the test.” It is:

The expected-value mechanism repeats the decision under test. What independent requirement would make the two disagree when that decision is wrong?

A boundary assertion derived from the inclusive contract answers that question. So can a small table of independently calculated invoice results. The implementation and the test necessarily share concepts such as money and eligibility; the oracle should not borrow the faulty decision procedure.

4.2 Surprise is conditional, not decorative

For intuition, let $K(T \mid I)$ denote the shortest description of a test T given implementation I . A copied oracle often has a short conditional description. But a large conditional description is not sufficient: arbitrary random bytes are difficult to predict and useless as an oracle. Furthermore, a

tiny assertion can rule out an important bug. Source-description length and behavioural evidence are different objects.

NCD is also symmetric, whereas “what does this test add, given this implementation and this suite?” is directional. A short oracle compared with a large implementation can receive a large distance simply because the sizes differ. Extracting comparable decision mechanisms mitigates that confound; it does not turn NCD into a conditional information estimator. The bridge from structural overlap to evidence is the reviewed oracle and its observed discrimination, not an identity between the two quantities.

A more direct model starts with a set $\mathcal{H}$ of plausible implementations, including faults of concern, and existing evidence E . A test yields an outcome O_T for each candidate. Under an explicitly chosen probability distribution over $\mathcal{H}$, its expected information gain is

$$\text{IG}(T \mid E) = H(\mathcal{H} \mid E) - \mathbb{E}_{O_T}[H(\mathcal{H} \mid E, O_T)].$$

Here entropy is over a hypothesis distribution, not over source tokens. This is a conceptual model, not a number the reference NCD tool estimates. In practice, a small mutation matrix can expose the distinctions a test actually makes without inventing a probability model.

A **sound** contract test separates unacceptable implementations from acceptable ones. Merely distinguishing two acceptable implementations by their private data layout is not useful evidence of correctness. Conversely, a discriminating but incorrect oracle is harmful. Specification provenance is therefore indispensable.

This makes the intended meaning of surprise precise: **a verdict that adds a relevant constraint not already supplied by the implementation’s own reasoning or by the existing suite.**

4.3 Three routes to independent constraints

Independent examples and boundaries. Derive expected values from a contract, a trusted external measurement or a small auditable calculation. For the discount rule, examine just below, exactly at and just above the threshold. Preserve these literals in the normalisation: the equality case is the point.

Metamorphic relations. When a general oracle is hard to compute, use relations the specification requires across executions. An aggregation result may be invariant to input permutation and additive over partitions. These properties take a different route from a copied loop. They still need anchors: a constant-zero implementation can satisfy several attractive algebraic relations. One independent nonzero example can eliminate that vacuity.

Temporal and effect contracts. Observe facts such as “a resource is released exactly once after a failing body” or “a failed transaction does not commit.” An event recorder can constrain order and ownership without reimplementing the production protocol. Comparing only return values misses these faults.

Independence is not a guarantee of correctness. A published standard can be misread, a reference implementation can share ancestry, and a metamorphic relation can be invalid for a domain. Record where the constraint comes from and why it applies.

4.4 Necessary counterexamples

A responsible application must include cases that defeat the slogan “higher NCD means a better test.”

- **Low distance, useful test.** A compact regression assertion can resemble the implementation yet catch a boundary mutation or a wiring error. A shared literal might be a mandatory protocol value, not accidental duplication.
- **High distance, useless test.** Adding irrelevant computations, random-looking literals or verbose fixture setup can increase structural novelty without changing any assertion or killing any additional fault. Comments alone will disappear under AST normalisation; use retained but behaviourally irrelevant syntax to demonstrate this attack honestly.
- **Shared operation, distinct observation.** `assert add(2, 3) == 5` may look like a trivial restatement, but can distinguish subtraction, truncation or a misrouted call. Whether it earns its maintenance cost depends on the plausible failures, not a ban on testing arithmetic.
- **Mirrored algorithm, residual value.** A duplicated oracle may still catch a fault introduced on only one side. Its distinctive weakness is correlated faults and coupled maintenance, not the logical impossibility of detecting any bug.

NCD therefore produces a request for examination, not an automatic quality verdict or licence to delete tests.

5. An executable demonstration

The repository contains a Python standard-library implementation, two before/after refactoring examples, and test-design examples spanning boundary oracles, algebraic properties and resource cleanup. The runner produces the exact NCD scores, normalised lengths, behaviour checks and fault-detection matrices used in the companion [results](#).

From the repository root:

```
python3 -m examples.run --output-dir results
python3 -m unittest discover -s tests -v
```

The checked-in results describe **constructed examples**, not a random sample of production systems. The demonstration asks bounded questions:

1. Does conservative normalisation make local-renaming duplicates identical?
2. Does a common-owner refactoring preserve the declared example contracts?
3. Can an implementation-mirroring oracle share a deliberately constructed fault?
4. Do independent boundary, metamorphic and effect constraints expose specified faults?
5. Can irrelevant novelty change a score without increasing fault detection?
6. Can a structurally similar test still detect a relevant mutation?

5.1 Observations in the accompanying run

Under CPython 3.13.15 and zlib 1.3.2 (level 9), the aggregation and resource-protocol pairs have alpha-normalised NCDs of 0.0993 and 0.0870 respectively. All four declared before/after value, validation and effect checks pass. These scores nominate the two mechanisms for examination; the checks establish only the enumerated example contracts.

The concrete boundary example concerns **integer shipping weights**: the fee is 5 through weight 10 and 8 above it. An erroneous `>= 10` implementation and its mirrored oracle return 8 at weight 10, while the independently specified result is 5. Their normalised distance is 0.0943. A separately contract-derived oracle using the first upper-tier integer, `>= 11`, has distance 0.1141 from the correct implementation and rejects the boundary fault. Both distances are low: their different evidential roles come from the contracts and verdicts, not a decisive score threshold.

The idempotence property for string canonicalisation admits a constant-empty implementation; the anchor `canonicalize(" A B ") == "a b"` rejects it. For resource publication, observing successful sends does not reveal the missing `finally`; observing a failed send constrains cleanup and exception identity. Across the three specified faulty implementations, at least one applicable test rejects each: 3/3 faults and 4/7 applicable test/fault observations. The unsound mirrored oracle is shown separately rather than counted as a valid baseline-passing test.

Appending comments to an oracle leaves its normalised representation unchanged (finite-compressor self-distance 0.0486). Adding retained but irrelevant literals and computation raises the distance between the original and padded oracle to 0.5494, while their verdicts on the correct shipping implementation and the boundary fault are identical. This is a concrete separation between structural novelty and incremental behavioural evidence.

Every mutation score must name its denominator. A test killing all three selected mutants has killed those three, not established a 100% real-world defect detection rate. Likewise, examples of poor or good rankings are observations about a chosen representation, compressor and corpus.

Implementation-specific normalisation and compression settings are documented in the repository README and the generated result metadata. Scores should be regenerated rather than copied into new claims under a different toolchain.

6. A practical joint workflow

Use two queues, not one global “similarity is bad” threshold.

Queue	Comparison	Review question	Acceptance evidence
Refactoring	Implementation vs implementation	Should this mechanism have one owner?	Independent contracts survive the refactor
Oracle audit	Oracle mechanism vs implementation	Is expected behaviour independently constrained?	A relevant correlated fault is distinguished
Suite growth	Candidate test vs existing tests, plus fault outcomes	What new constraint does this add?	New fault or required scenario covered

For suite growth, compare fault-detection sets. If $D(T)$ is the set of specified faults detected by T , then

$$D(T) \setminus \bigcup_{U \in S} D(U)$$

is the observed incremental fault coverage relative to suite S . An empty set does not prove redundancy beyond that fault model: tests can cover distinct environments, integrations or historical regressions absent from the matrix. It does provide a concrete question about maintenance value.

The workflow also improves large refactors. Structurally mirrored tests tend to demand matching rewrites, whereas externally specified invariants can remain stable while implementation structure changes. This is a reason to prefer independent constraints, not a promise that every black-box test is sufficient.

7. Evaluation agenda and limitations

A larger study should separate three tasks: clone retrieval, correlated-oracle identification and incremental test selection. Measure each against the appropriate outcome rather than using one NCD ranking as its own ground truth.

For clone retrieval, label whether a shared owner was beneficial, including policy-divergence negatives. For oracle audits, use independently reviewed fault/contract pairs, real historical defects and deliberately correlated mistakes. For test selection, evaluate held-out faults, preserve mandatory contracts, and compare against coverage, mutation selection, token similarity and unnormalised compression baselines.

Freeze thresholds on a development corpus and report held-out precision and recall. Stratify by language, unit size, boilerplate fraction, syntax kind and oracle style. Vary compressors and normalisation policies. Review decisions without showing reviewers the score first, where practical. Do not optimise novelty against the same hand-picked mutants used to claim success.

Important limitations include lossy normalisation, compressor overhead and window size, sensitivity to unit boundaries, external dependency semantics, dynamically resolved names, incomplete mutation models and incorrect specifications. Source syntax also omits oracle provenance: two identical constants may have been copied from an implementation or independently derived from a standard. Compression cannot recover that history.

There is no theorem here that low NCD implies a bad test, nor that high NCD implies a good one. The supported design principle is narrower and more useful: **look for repeated implementation information when deciding what to consolidate, and look for independent behavioural information when deciding what to trust.**

8. Conclusion

Alpha-normalised NCD turns arbitrary local naming into a less distracting coordinate system for structural comparison. It can reveal one mechanism wearing different names and one implementation wearing the costume of its own oracle.

The engineering actions differ. Refactoring consolidates a mechanism under one owner. Test design introduces a constraint with an independent source. Good tests are surprising in that second sense: they can tell the implementation something it did not already assume.

References and provenance

The [bibliography](#), also included in the PDF, provides primary references and claim-level context. This note develops the alpha-normalised NCD and anti-mirroring testing principles in Hara’s software-engineering practice. AI assistance was used to draft the exposition and reference implementation; the executable results identify what was actually measured. Publication is a research note on GitHub, not a claim of peer review.

Bibliography

1. Ming Li, Xin Chen, Xin Li, Bin Ma, and Paul M. B. Vitányi. 2004. **The Similarity Metric.** *IEEE Transactions on Information Theory* 50(12), 3250–3264. [doi:10.1109/TIT.2004.838101](#). [Author PDF](#). Foundational normalized information distance and its compression approximation; theoretical properties of NID do not automatically transfer to a particular compressor or lossy source representation.
2. Rudi Cilibrasi and Paul M. B. Vitányi. 2005. **Clustering by Compression.** *IEEE Transactions on Information Theory* 51(4), 1523–1545. [doi:10.1109/TIT.2005.844059](#). [Corrected author version](#). Defines and applies practical NCD; motivates careful treatment of compressor properties and compressed lengths.
3. Brenda S. Baker. 1995. **On Finding Duplication and Near-Duplication in Large Software Systems.** *Proceedings of the Second Working Conference on Reverse Engineering*, 86–95. [doi:10.1109/WCRE.1995.514697](#). Established clone detection under systematic substitution of identifiers and constants, with reengineering applications.
4. Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. 2015. **The Oracle Problem in Software Testing: A Survey.** *IEEE Transactions on Software Engineering* 41(5), 507–525. [doi:10.1109/TSE.2014.2372785](#). [Author PDF](#). Separates producing test inputs from deciding whether observed behaviour is correct, and surveys sources of oracles.
5. Huai Liu, Fei-Ching Kuo, Dave Towey, and Tsong Yueh Chen. 2014. **How Effectively Does Metamorphic Testing Alleviate the Oracle Problem?** *IEEE Transactions on Software Engineering* 40(1), 4–22. [doi:10.1109/TSE.2013.46](#). [Author manuscript](#). Studies specification relations across executions as partial oracles; distinct-looking test syntax does not necessarily imply distinct relations.
6. Yue Jia and Mark Harman. 2011. **An Analysis and Survey of the Development of Mutation Testing.** *IEEE Transactions on Software Engineering* 37(5), 649–678. [doi:10.1109/TSE.2010.62](#). Reviews fault-based assessment through introduced program changes and test discrimination.
7. Robert Feldt, Richard Torkar, Tony Gorschek, and Wasif Afzal. 2008. **Searching for Cognitively Diverse Tests: Towards Universal Test Diversity Metrics.** *ICST Workshops*. [doi:10.1109/ICSTW.2008.36](#). [Author PDF](#). Proposes NCD-based test diversity and compares it with human test clustering.
8. Robert Feldt, Simon Poulding, David Clark, and Shin Yoo. 2016. **Test Set Diameter: Quantifying the Diversity of Sets of Test Cases.** *ICST*, 223–233. [doi:10.1109/ICST.2016.33](#). [Paper](#). Extends compression-derived diversity to multisets and evaluates test selection on four systems.

9. Takashi Ishio, Naoto Maeda, Kensuke Shibuya, and Katsuro Inoue. 2018. **Cloned Buggy Code Detection in Practice Using Normalized Compression Distance**. *ICSME*, 148–151. [doi:10.1109/ICSME.2018.00022](https://doi.org/10.1109/ICSME.2018.00022). [Author PDF](#). Applies NCD to retrieval of clones of known faulty code in industrial use.
10. Islam T. Elgendy, Robert M. Hierons, and Phil McMinn. 2024. **Evaluating String Distance Metrics for Reducing Automatically Generated Test Suites**. *AST*, 171–181. [doi:10.1145/3644032.3644455](https://doi.org/10.1145/3644032.3644455). [Author PDF](#). Directly compares textual test-source distances, including NCD, for suite reduction and evaluates mutation-score retention.

These works establish the component techniques and prior applications. This note’s focus is the combined alpha-normalised review workflow and its distinction between structural overlap and independently sourced behavioural evidence.

Constructed examples

Measured with CPython 3.13.15, zlib 1.3.2, level 9, wbits=15; symmetric minimum-concatenation NCD, unclipped.

These are illustrative fixtures, not empirical validation of NCD as a quality metric. Low structural distance is an attention signal; only the explicit behavior contracts justify a refactor or regression test.

Refactoring contracts

- aggregation values: pass
- aggregation policy errors: pass
- resource values and effects: pass
- resource failure cleanup: pass

Similarity (zlib level 9)

Pair	Raw NCD	Alpha-normalized NCD
renamed aggregation	0.2235	0.0993
renamed resource protocol	0.1780	0.0870
boundary regression	0.2051	0.0559
mirrored faulty oracle	0.2650	0.0943
contract derived oracle	0.4841	0.1141
padded comments	0.9658	0.0386
padded oracle comments	0.9488	0.0486
padded oracle computation	0.6730	0.5494

The comments pair adds only textual surprise; the padded-oracle pair adds retained but candidate-irrelevant literals and computation, increasing measured structural distance without changing verdicts. The boundary pair changes `>` to `>=`; the independent integer-contract oracle uses `>= 11` and rejects that fault despite structural resemblance.

Three independent-contract mutation scenarios

Fault	Test	Baseline	Fault
boundary greater or equal	independent nonboundary examples	pass	survived
boundary greater or equal	independent boundary examples	pass	killed
boundary greater or equal	contract derived oracle	pass	killed
constant canonicalizer	idempotence	pass	survived
constant canonicalizer	anchored example	pass	killed
missing failure cleanup	successful effects	pass	survived
missing failure cleanup	failure cleanup and exception identity	pass	killed

Irrelevant novelty: identical verdicts

Candidate	Original oracle	Padded oracle
baseline	pass	pass
boundary fault	reject	reject

3/3 injected faults detected; 4/7 applicable test/fault observations killed. Outside the independent-contract matrix, an intentionally unsound mirrored oracle and faulty implementation *agree* at weight 10 (both return 8); the external contract says 5. Idempotence alone admits a constant function; the anchor excludes it. Success-only testing misses cleanup after a failed send.

The test fixtures are intentionally small and selected. Neither these ratios nor NCD scores estimate field defect rates.